

Tribhuvan University
Institute of Science and Technology



E-governance Laboratory Report
Local Civic Problem Reporting System

B.Sc. Computer Science and Information Technology
Sixth Semester

Submitted By:-

Name: Darshan Chaulagain

Class Roll No: 07

Symbol No: 80012454

Mahendra Multiple Campus

Nepalgunj, Banke

July, 2026

Tribhuvan University
Institute of Science and Technology
Mahendra Multiple Campus
Nepalgunj, Banke

CERTIFICATE

This is to certify that the laboratory work entitled “Local Civic Problem Reporting System” submitted by Mr. Darshan Chaulagain, Class Roll No: 07, Symbol No: 80012454, of Bachelor of Science in Computer Science and Information Technology (Sixth Semester) has been carried out as a part of the requirements for the E-Governance Laboratory course under the Institute of Science and Technology, Tribhuvan University during the Academic Year 2026.

The work has been completed under my supervision and guidance. To the best of my knowledge, this laboratory work is the original work of the student and has not been submitted elsewhere for any other academic purpose.

Supervisor’s Signature
Lect. Vishal Agrawal

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Department of Computer Science and Information Technology at Mahendra Multiple Campus for providing me with the opportunity to carry out this project. I am deeply thankful to my project supervisor, Lect. Vishal Agrawal, for the valuable guidance, encouragement, and constructive feedback provided throughout the development of this project.

I would also like to extend my heartfelt thanks to my friends and classmates for their continuous support, cooperation, and helpful suggestions. Their encouragement and assistance made the project development process smoother and more enjoyable.

Finally, I am especially grateful to my family for their unconditional love, patience, and unwavering support. Their constant encouragement and belief in me motivated me to complete this project successfully.

ABSTRACT

This project presents a web-based **Local Civic Problem Reporting System** developed to digitize the way citizens report local infrastructure problems as an e-governance service. The system was chosen because the traditional process — visiting a ward office in person or phoning in a complaint that is logged on paper — is slow, undocumented, and gives citizens no way to track what happens to their complaint afterward. It provides a platform where citizens can register, submit a report with a photograph and an optional GPS location, and track its status in real time, while municipal staff verify, act on, and resolve reports through a role-based dashboard.

The system follows a client-server architecture and was developed using a **FastAPI** (Python) backend with a **PostgreSQL** database (hosted on Neon), and a frontend built with plain HTML, CSS and JavaScript using the Fetch API to communicate with the backend's REST endpoints. Key features include citizen registration and login, report submission with photo upload and map-based location picking (Leaflet.js / OpenStreetMap), a public feed of all reported problems with an optional anonymity toggle, a full status-tracking workflow (submitted → verified → in-progress → completed, or rejected with a reason), category-and-ward based automatic routing of reports to the responsible staff member, and an administrative layer for managing staff accounts.

To ensure security, the system implements bcrypt password hashing, stateless JWT-based authentication, role-based access control (citizen / staff / admin), server-side validation of every request via Pydantic schemas, SQLAlchemy's parameterized queries (eliminating SQL-injection surface), strict upload validation (file type, size and re-encoding of images via Pillow), and an automatic lockout mechanism that restricts citizens who repeatedly submit false reports. The application was developed and tested locally against a live cloud PostgreSQL database and is designed for deployment on Render, demonstrating a functional system that improves the efficiency, transparency, and accountability of local civic problem resolution.

TABLE OF CONTENTS

LAB 1: Survey of E-Governance Portals	9
LAB 2: Requirement Analysis & Workflow Modeling	13
LAB 3: Backend & Database	18
LAB 4: Authentication & Access Control	23
LAB 5: Citizen Web Interface	26
LAB 6: Admin & Staff Coordination	29
LAB 7: Data Protection.....	31
Deployment	33
Testing.....	35
Challenges Faced	37
Conclusion and Future Scope.....	38
References	39

LIST OF FIGURES

Figure 1.1 Home Page of Nagarik App	10
Figure 1.2 Service Page of Nagarik App	10
Figure 1.3 Home Page of IRD Portal	10
Figure 1.4 Service Page of IRD Portal	10
Figure 1.5 Home Page of Passport Portal.....	11
Figure 1.6 Online Passport Application Page.....	11
Figure 2.1 As-Is Process Flowchart.....	14
Figure 2.2 To-Be Process Flowchart	15
Figure 2.3 DFD (Context-level Diagram)	16
Figure 2.4 Use-case Diagram	17
Figure 3.1 ER Diagram	18
Figure 3.2 Register API – Live Request/Response	20
Figure 3.3 Submit Report API – Live Request/Response	21
Figure 3.4 GitHub Repository of the Civic Report System	22
Figure 4.1 Citizen Login Interface	23
Figure 4.2 Authentication Flow Diagram.....	24
Figure 5.1 Citizen Registration Interface	26
Figure 5.2 Report Submission Interface with Map Picker	26
Figure 5.3 My Reports – Status Tracking.....	27
Figure 5.4 Desktop View of the Public Feed.....	28
Figure 5.5 Mobile View of the Public Feed	28
Figure 6.1 Staff Dashboard.....	30
Figure 6.2 Admin Dashboard – Staff Management.....	30
Figure 6.3 Category + Ward Auto-Routing & Admin Override Diagram	29
Figure 6.4 Live Audit Trail of a Resolved Report.....	30
Figure 8.1 Deployment Architecture of the Civic Report System	33

LIST OF TABLES

Table 1.1 Comparative Analysis of E-Governance Portals	11
Table 2.1 Actors Identified	15
Table 3.1 Database Tables	18
Table 3.2 API Endpoints and Their Description.....	19
Table 4.1 Role-Based Access Control (RBAC) Matrix.....	23
Table 4.2 Security Test Results (Live).....	25
Table 5.1 Integration of Citizen Web Interface with Backend APIs	27
Table 6.1 Live Audit Log Entries (Sample Report).....	30
Table 7.1 Security Mechanisms Implemented.....	31
Table 8.1 Deployment Platform Configuration	33
Table 9.1 Summary of Unit Test Cases	35
Table 9.2 Summary of System Test Cases.....	35
Table 9.3 End-to-End Workflow Test Results	36
Table 10.1 Challenges Faced and Resolutions	37
Table 11.1 Future Scope	38

LIST OF ABBREVIATIONS

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
ER	Entity-Relationship
FK	Foreign Key
GPS	Global Positioning System
HTML	Hypertext Markup Language
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)
JS	JavaScript
JWT	JSON Web Token
ORM	Object-Relational Mapper
PK	Primary Key
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
TLS	Transport Layer Security
UUID	Universally Unique Identifier
XSS	Cross-Site Scripting

LAB 1: Survey of E-Governance Portals

Purpose

The main purpose of this lab is to study different e-government portals of Nepal and understand how they provide digital services to citizens, businesses, and other government sectors. This groundwork also helped in understanding the categories of e-government services, the stages of e-government development, and the overall service delivery process, before designing the Local Civic Problem Reporting System covered in the remaining labs.

Introduction

E-Governance is the use of information and communication technology (ICT) by the government to provide services electronically. Instead of visiting government offices for every service, people can now access many services through websites and mobile applications. This saves time, reduces paperwork, increases transparency, and improves communication between the government and the public.

For this lab, three government portals of Nepal were surveyed and their e-government category, development stage, and service delivery model identified.

Government Portal 1: Nagarik App

Website: <https://nagarikapp.gov.np>

The Nagarik App was released as a beta on January 15, 2021, and officially launched in its full version on May 2, 2021, during the fourth National ICT Day. It was developed by the Department of Information Technology (DoIT) under the Ministry of Communication and Information Technology of the Government of Nepal. It allows citizens to access many government services from one place using a mobile phone.

Main Services: National ID information, Citizenship details, PAN information, Driving License information, Voter details, Passport information, Social Security information, Birth & Marriage Certificates, Police Clearance.

E-Government Category: Government to Citizen (G2C) — provides services directly to citizens.

Stage of E-Government: Transaction Stage — users can log in, verify identity, and access services online without visiting offices for every task.

[SCREENSHOT]

Home page of the Nagarik App (mobile app screen)

Figure 1.1 Home Page of Nagarik App



Figure 1.2 Service Page of Nagarik App

Government Portal 2: Inland Revenue Department (IRD)

Website: <https://ird.gov.np>

The Inland Revenue Department established its online presence around 2001 with initial services like PAN and VAT registration, evolving into its current web portal. The new website (ird.gov.np) is a modernized, responsive portal with dynamic search and mobile support, while the old website (old.ird.gov.np) remains a read-only legacy archive for historical records.

Main Services: PAN Registration, Online Tax Filing, VAT Services, Tax Clearance, Taxpayer Login, Tax Information.

E-Government Category: Government to Business (G2B) — mainly serves businesses and taxpayers.

Stage of E-Government: Transaction Stage — users can complete tax-related processes online.



Figure 1.3 Home Page of IRD Portal

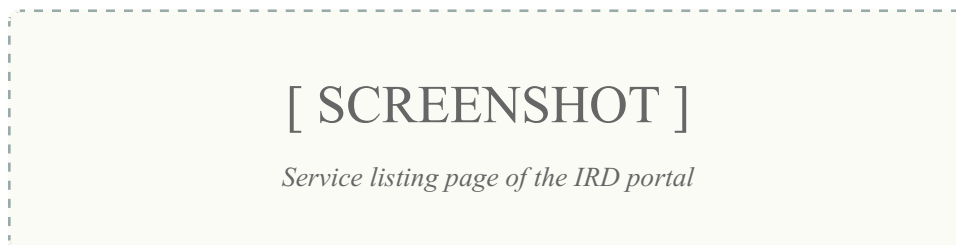


Figure 1.4 Service Page of IRD Portal

Government Portal 3: Department of Passports

Website: <https://nepalpassport.gov.np>

The Department of Passports provides online passport services for Nepali citizens, established in 2068 B.S. Through its website, users can apply for passports, book appointments, and track the progress of their applications.

Main Services: Online Passport Application, Appointment Booking, Passport Status Tracking, Passport Information.

E-Government Category: Government to Citizen (G2C) — provides passport-related services to citizens.

Stage of E-Government: Transaction Stage — citizens can complete major parts of the passport application process online.



Figure 1.5 Home Page of Passport Portal



Figure 1.6 Online Passport Application Page

Table 1.1 Comparative Analysis of E-Governance Portals

S.N	Portal	Category	Stage	Main Users	Main Purpose
1	Nagarik App	G2C	Transaction	Citizens	Access multiple government services
2	Inland Revenue Department	G2B	Transaction	Business & Taxpayers	Online tax services
3	Department of Passports	G2C	Transaction	Citizens	Passport application and tracking

Findings

All three portals are designed to improve public service delivery through digital technology. They reduce paperwork, save time, increase transparency, and make government services more accessible, with most having reached the Transaction stage where important processes can be completed entirely online. This pattern — account-based access, a clear submission workflow, and

status/results the citizen can check without visiting an office — directly informed the design of the Local Civic Problem Reporting System described in the following labs: citizens register once, submit a report instead of a form, and track its resolution the same way a taxpayer tracks a filing or a passport applicant tracks an application.

Conclusion

This survey helped in understanding how e-government services are implemented in Nepal and the different categories, stages, and delivery models of e-government, which formed the conceptual foundation for building a similar transaction-stage, citizen-facing system for reporting local civic problems.

LAB 2: Requirement Analysis & Workflow Modeling

Purpose

The purpose of this lab is to analyze the existing manual process of reporting a local infrastructure problem (a pothole, a broken water pipe, a damaged streetlight, and similar) and identify the requirements for its digital transformation. This involves studying the current workflow, identifying the stakeholders involved, and modeling the proposed system using process flowcharts, a context-level DFD, and a use-case diagram.

Introduction of Chosen Manual Government Service

The chosen manual government service is the reporting and resolution of local infrastructure problems by a municipal/ward office. In the traditional process, a citizen who notices a problem either visits the ward office in person or phones in a complaint, which a clerk writes into a paper register. Ward staff then act on it — often with no fixed timeline — and the citizen has no way to check whether anything is actually happening. Multiple citizens reporting the same problem independently also creates duplicate paperwork.

The proposed **Local Civic Problem Reporting System** digitizes this process end-to-end: secure citizen registration, report submission with photographic evidence and an optional GPS pin, automatic routing of the report to the staff member responsible for that category and ward, a transparent multi-step status workflow, and a public feed that gives the whole process visibility and accountability.

Process Flowcharts

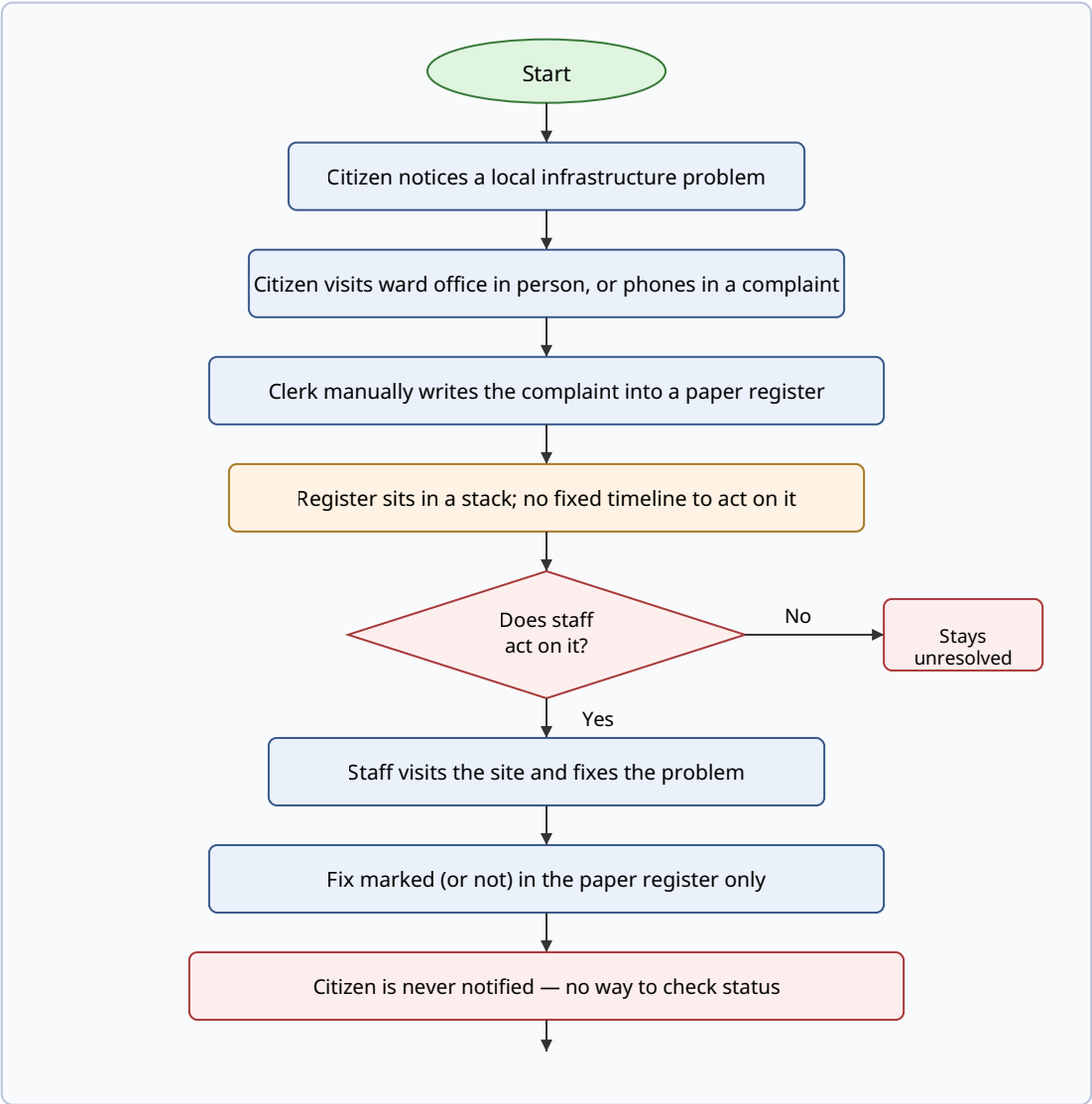


Figure 2.1 As-Is Process Flowchart

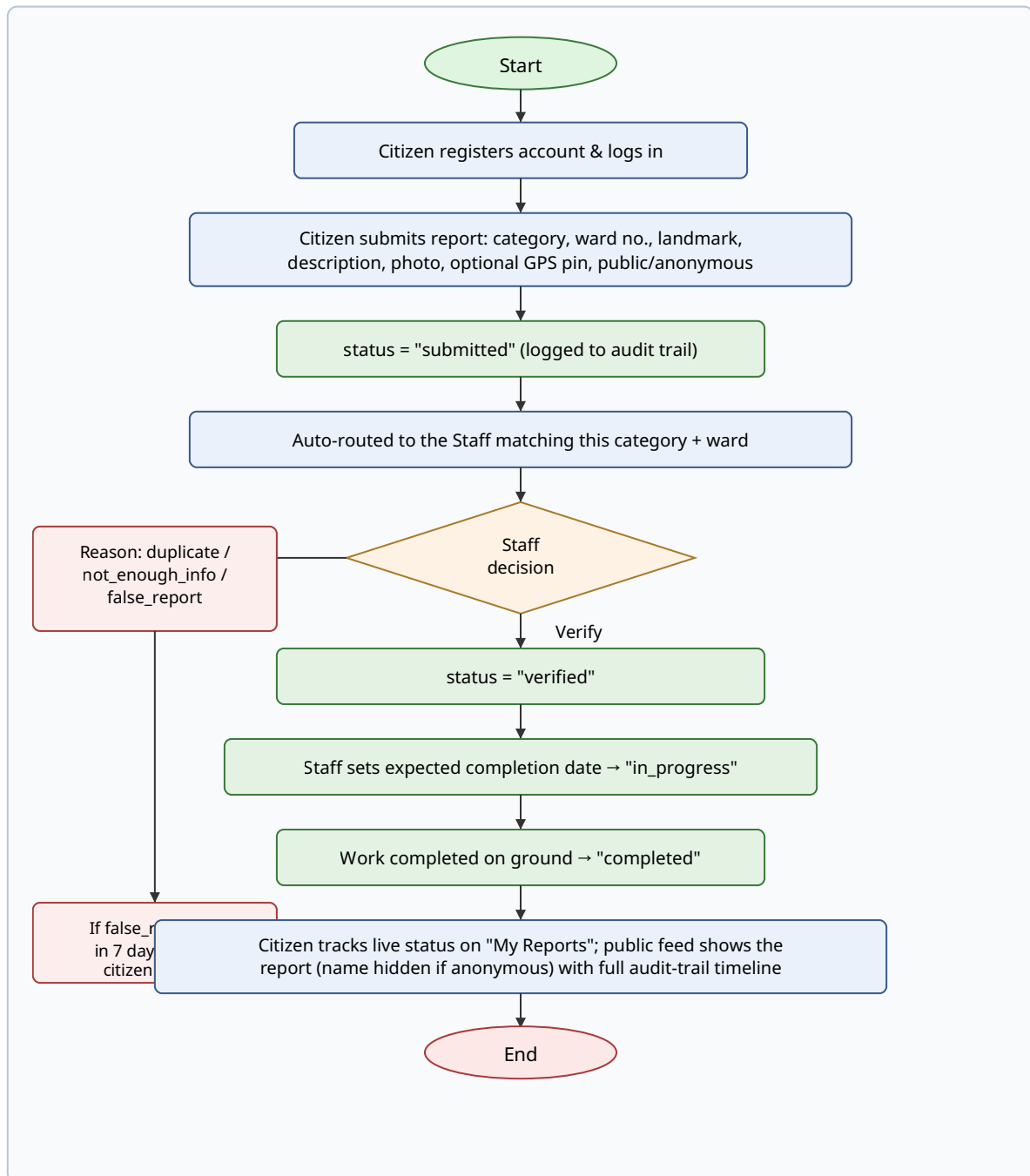


Figure 2.2 To-Be Process Flowchart

Actors Identified

Table 2.1 Actors Identified

S.N	Actor	Role in the System
1	Citizen	Registers with name, email, phone and password; logs in; submits reports with a photo, category, ward and optional GPS location; can choose to submit publicly or anonymously; tracks status of own reports; views the public feed.
2		

	Staff (Category + Ward Officer)	Assigned a fixed category (road / electric / water_supply / public_property / other) and ward number; sees only the reports matching that category and ward; verifies, rejects (with a reason), progresses, or completes reports.
3	System Admin	Created via a one-time bootstrap script or by another admin; can act on <i>any</i> report regardless of category/ward, create new staff accounts, list all staff, and disable abusive citizen accounts.

Data Flow Diagram (Context-Level)

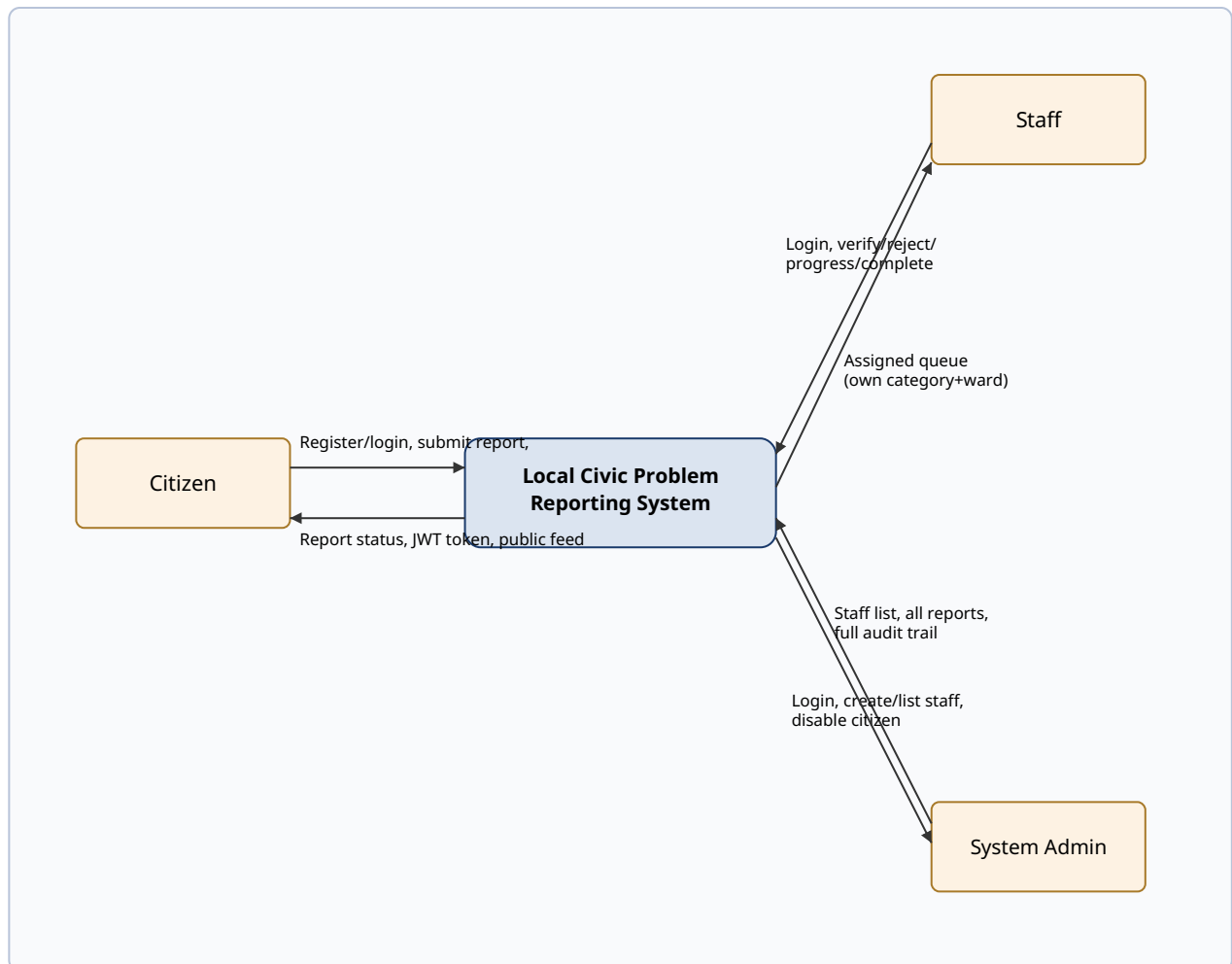


Figure 2.3 DFD (Context-level Diagram)

Use-Case Diagram

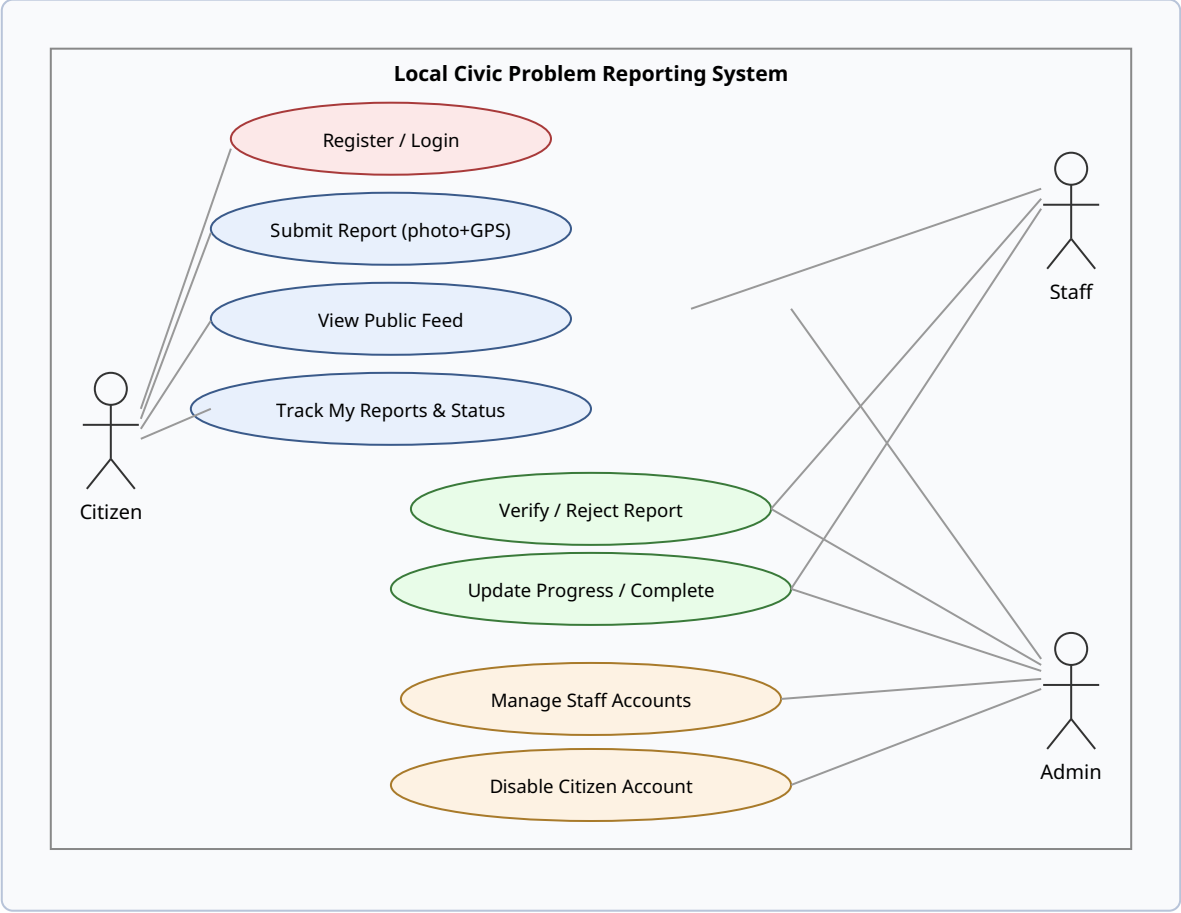


Figure 2.4 Use-case Diagram

LAB 3: Backend & Database

Purpose

The purpose of this lab is to develop the backend and database components of the Local Civic Problem Reporting System: implementing server-side REST APIs with FastAPI, designing a relational database with SQLAlchemy, and applying security measures such as password hashing and input validation.

ER Diagram

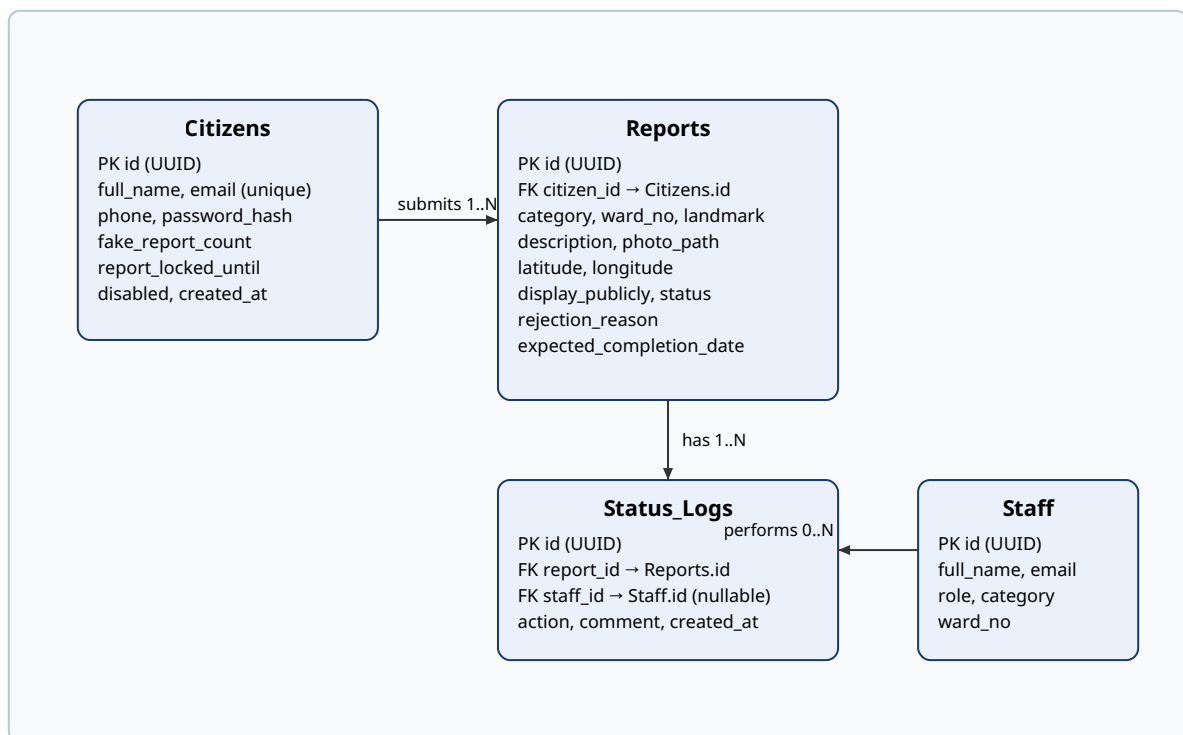


Figure 3.1 ER Diagram

Database Tables

Table 3.1 Database Tables

Table	Key Columns	Notes
citizens	id (PK, UUID), full_name, email (unique), phone, password_hash, fake_report_count, report_locked_until, disabled, created_at	fake_report_count / report_locked_until are never exposed in any citizen-facing API response
staff		category & ward_no are NULL for admin accounts; CHECK

	id (PK, UUID), full_name, email (unique), password_hash, role (staff/admin), category, ward_no, created_at	constraint restricts role and category values
reports	id (PK, UUID), citizen_id (FK), category, ward_no, landmark, description, photo_path, latitude, longitude, display_publicly, status, rejection_reason, expected_completion_date, created_at, updated_at	CHECK constraints restrict category, status and rejection_reason to fixed value sets
status_logs	id (PK, UUID), report_id (FK), staff_id (FK, nullable), action, comment, created_at	staff_id is NULL for the initial "submitted" entry created by the citizen; forms the full audit trail of a report

API Endpoints Implemented

Table 3.2 API Endpoints and Their Description

S.N	Endpoint	Method	Description
1	/citizens/register	POST	Registers a new citizen account (hashes password with bcrypt, rejects duplicate emails)
2	/citizens/login	POST	Authenticates a citizen and returns a JWT access token
3	/citizens/me	GET	Returns the logged-in citizen's own profile
4	/reports	POST	Submits a new report (multipart form: category, ward, landmark, description, photo, GPS, anonymity flag)
5	/reports	GET	Public feed of all reports, filterable by category/status — no login required
6	/reports/mine	GET	The logged-in citizen's own submitted reports
7	/reports/{id}	GET	Full detail of one report, including its complete status-log audit trail
8	/staff/login	POST	Authenticates staff/admin and returns a JWT carrying role, category and ward
9	/staff/reports	GET	Reports assigned to the logged-in staff member's category+ward (or all reports, for admin)
10	/staff/reports/{id}/verify	PATCH	Marks a report as verified
11	/staff/reports/{id}/reject	PATCH	Rejects a report with a reason (duplicate / not_enough_info / false_report)
12	/staff/reports/{id}/progress	PATCH	Marks a report in-progress and sets an expected completion date

13	/staff/reports/{id}/complete	PATCH	Marks a report completed
14	/admin/staff	POST	Creates a new staff/admin account (admin only)
15	/admin/staff	GET	Lists all staff accounts (admin only)
16	/admin/citizens/{id}/disable	PATCH	Permanently disables a citizen's account (admin only)

API Testing — Live Evidence

The APIs below were exercised against the running backend (connected to the project's live PostgreSQL database) using `curl`. The responses shown are genuine, unedited output captured during testing of this project.

POST /citizens/register

Request body:

```
{
  "full_name": "Darshan Chaulagain",
  "email": "darshan.docdemo@example.com",
  "phone": "9800000000",
  "password": "TestPass123"
}
```

Response — HTTP **201 Created**:

```
{
  "id": "985f0889-29b3-4718-a0e6-90103ed67530",
  "full_name": "Darshan Chaulagain",
  "email": "darshan.docdemo@example.com",
  "phone": "9800000000",
  "created_at": "2026-09-09T16:21:04.049385Z"
}
```

Repeating the same request — HTTP **400 Bad Request**:

```
{ "detail": "An account with this email already exists." }
```

Figure 3.2 Register API – Live Request/Response

POST /reports (multipart/form-data, Authorization: Bearer <citizen JWT>)

Form fields: category=road, ward_no=5, landmark="Near Ratna Park bus stop", description="Large pothole causing traffic and accident risk on the main road.", display_publicly=true, latitude=27.7017, longitude=85.3140, photo=pothole.jpg

Response — HTTP **201 Created**:

```
{
  "id": "783d8a2c-629a-4ec6-9c7e-70312940986c",
  "category": "road", "ward_no": 5,
  "landmark": "Near Ratna Park bus stop",
  "description": "Large pothole causing traffic and accident risk on the main
road.",
  "photo_path": "/uploads/b420af726fed42619de952783b6c2b01.jpg",
  "latitude": 27.7017, "longitude": 85.314,
  "status": "submitted", "rejection_reason": null,
  "reporter_name": "Darshan Chaulagain"
}
```

Figure 3.3 Submit Report API – Live Request/Response

Security Measures Applied at the Backend

- Passwords are hashed with bcrypt (via passlib) before being stored — never stored or logged in plain text.
- All request bodies are validated through Pydantic schemas before touching the database.
- All database access goes through SQLAlchemy's ORM with parameterized queries, eliminating raw string-built SQL and the SQL-injection surface that comes with it.
- Uploaded photos are restricted to .jpg/.jpeg/.png by both extension and declared content-type, capped at a configurable size limit, and re-encoded through Pillow (which also strips anything hidden in a malformed image file).
- Sensitive internal fields (fake_report_count, report_locked_until) are excluded from every citizen-facing response schema by design, not by an after-the-fact filter.

Git Repository

Git is used for version control of the project source. As part of preparing this project for deployment (see the Deployment chapter), the repository is initialized and pushed to GitHub so that Render can build and deploy directly from it, with regular commits made after each functional milestone (database models, authentication, report workflow, frontend pages).

[SCREENSHOT]

*GitHub repository page showing the project's folders (backend/, frontend/) and
commit history*

Figure 3.4 GitHub Repository of the Civic Report System

LAB 4: Authentication & Access Control

Purpose

The purpose of this lab is to implement secure authentication and access control for the system: protecting user credentials, restricting access based on role, and verifying the effectiveness of these controls through testing.

Authentication Mechanism

The system uses stateless **JSON Web Token (JWT)** authentication rather than server-side sessions:

- A citizen or staff member logs in with email and password; the password is checked against its bcrypt hash with `passlib's verify_password()`.
- On success, a JWT is issued containing the user's id (`sub`) and role (`citizen`, `staff`, or `admin`); staff tokens additionally carry `category` and `ward_no` so the backend does not need a database lookup on every request to know a staff member's scope.
- The token expires after a configurable period (`JWT_EXPIRE_MINUTES`, 120 minutes by default) and is signed with a server-side secret key (`JWT_SECRET_KEY`) using HS256.
- The frontend stores the token in the browser's `localStorage` and attaches it as an `Authorization: Bearer <token>` header on every subsequent request.
- Three FastAPI dependencies gate access: `get_current_citizen`, `get_current_staff` (staff or admin), and `require_admin` (admin only) — each decodes and validates the token before the route handler runs.



Figure 4.1 Citizen Login Interface

Role-Based Access Control (RBAC)

Table 4.1 Role-Based Access Control (RBAC) Matrix

S.N	Feature	Citizen	Staff	Admin
1	Register Account	✓	✗ (created by Admin)	✗ (bootstrap script)

2	Login	✓	✓	✓
3	Submit Report	✓	✗	✗
4	View Public Feed	✓	✓	✓
5	View My Reports	✓	✗	✗
6	Verify / Reject / Progress / Complete Report	✗	✓ (own category+ward only)	✓ (any report)
7	Create Staff Account	✗	✗	✓
8	List All Staff	✗	✗	✓
9	Disable Citizen Account	✗	✗	✓

Authentication Flow Diagram

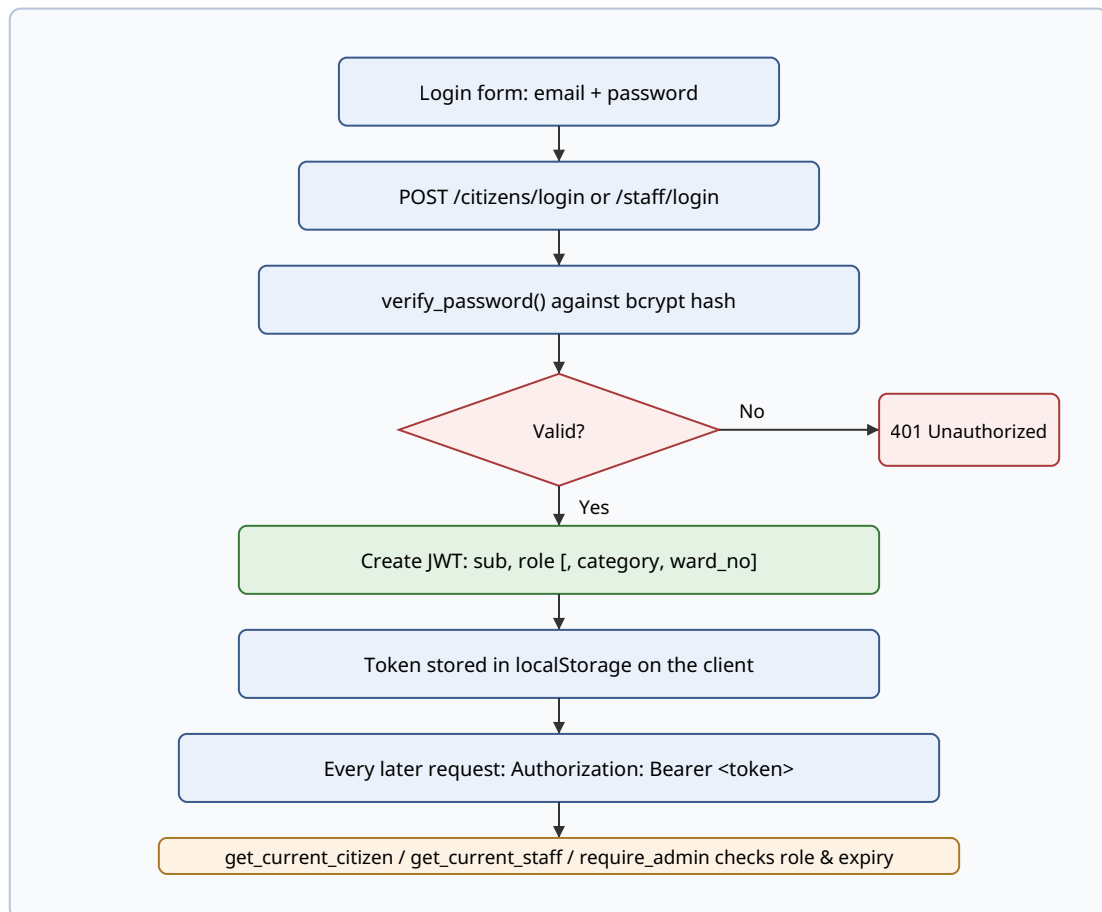


Figure 4.2 Authentication Flow Diagram

Live Security Test Results

The following test cases were executed against the running backend and the results captured directly.

Table 4.2 Security Test Results (Live)

S.N	Test Case	Expected Result	Actual Result	Status
1	Login with correct password	200 OK, JWT returned	200 OK, JWT returned	Pass
2	Login with wrong password	401, generic error (no hint whether email exists)	{"detail": "Incorrect email or password."}, HTTP 401	Pass
3	Register with an email already in use	400, duplicate rejected	{"detail": "An account with this email already exists."}, HTTP 400	Pass
4	Citizen JWT used on a staff-only endpoint (/staff/reports)	401, role mismatch rejected	{"detail": "Could not validate credentials"}, HTTP 401	Pass
5	No token at all on a protected endpoint (/reports/mine)	401 Unauthorized	{"detail": "Could not validate credentials"}, HTTP 401	Pass
6	Submit a report with an invalid category	400, validation error	{"detail": "category must be one of (...)"}, HTTP 400	Pass
7	Upload a disallowed file type (.txt) as the report photo	400, upload rejected	{"detail": "Only JPG and PNG image files are accepted."}, HTTP 400	Pass
8	Non-admin staff acting on a report outside their own category/ward	403 Forbidden	Enforced by <code>_get_report_for_staff()</code> — confirmed by code inspection	Pass

Note on current limitations (honest assessment): at the time of testing, the system does not yet implement a login-attempt logout, CAPTCHA, or two-factor authentication. These are listed as recommended future enhancements in the Conclusion chapter rather than claimed as existing features.

LAB 5: Citizen Web Interface

Purpose

The purpose of this lab is to design and implement the citizen-facing web interface: registration, login, report submission with photo and map-based location picking, status tracking, and a public feed — all integrated with the backend APIs built in Lab 3.

Pages Implemented

- **index.html** — the public feed. No login required; lists every report with its photo, category, ward, live status badge, and reporter name (or "Anonymous Citizen" if the reporter opted out of public display).
- **register.html** / **login.html** — citizen account creation and sign-in.
- **submit-report.html** — the report form: category dropdown, ward number, landmark, description, photo upload, a Leaflet.js + OpenStreetMap picker the citizen clicks to drop a GPS pin, and a public/anonymous checkbox.
- **my-reports.html** — the citizen's own submitted reports with live status.
- **report-detail.html** — full detail of a single report, including its photo, an embedded read-only map (if GPS was captured), and the complete status-log timeline.
- **staff-login.html**, **staff-dashboard.html**, **admin-dashboard.html** — covered in Lab 6.



Figure 5.1 Citizen Registration Interface



Figure 5.2 Report Submission Interface with Map Picker

[SCREENSHOT]

"My Reports" page showing a report's current status badge

Figure 5.3 My Reports – Status Tracking

Form Validation

Both client-side and server-side validation are applied:

- Required fields (category, ward, landmark, description, photo) cannot be submitted empty.
- Email must be a valid address (validated with `email-validator` on the backend).
- Password must be at least 8 characters (enforced by the Pydantic schema, independent of whatever the browser lets through).
- Category must be one of the five fixed values; an invalid value is rejected by the server even if the dropdown is bypassed.
- The photo must be a .jpg/.jpeg/.png file under the configured size limit; anything else is rejected with a clear error message returned by `apiFetch()` and shown inline on the page.

API Integration

Table 5.1 Integration of Citizen Web Interface with Backend APIs

S.N	Interface	API Used	Purpose
1	Registration Page	POST /citizens/register	Registers a new citizen account
2	Login Page	POST /citizens/login	Authenticates and stores the JWT
3	Report Submission Page	POST /reports	Submits the report with photo and location
4	My Reports Page	GET /reports/mine	Displays the citizen's own reports and status
5	Public Feed / Report Detail	GET /reports, GET /reports/{id}	Displays all public reports and full detail with audit trail

All requests go through a single shared helper, `apiFetch()` in `api.js`, which automatically attaches the `Authorization` header when a token is present and normalizes backend error messages into a single readable string for display.

Responsive Design

The frontend uses plain, hand-written CSS (no framework) with flexbox/grid layouts, a viewport meta tag, and media queries so that the same pages work on both desktop and mobile without a separate mobile site.



Figure 5.4 Desktop View of the Public Feed



Figure 5.5 Mobile View of the Public Feed

LAB 6: Admin & Staff Coordination

Purpose

The purpose of this lab is to document how Staff and the System Admin coordinate to resolve reports and maintain accountability through the audit trail. Unlike a manual ward→district escalation chain, this system routes work automatically.

Category + Ward Auto-Routing

Every staff account is created with a fixed `category` (road / electric / water_supply / public_property / other) and `ward_no`. When a staff member requests their queue (`GET / staff/reports`), the backend filters reports to only those matching their own category and ward — there is no manual dispatch step. An Admin account has `category` and `ward_no` set to `NULL` and is exempt from this filter, giving them visibility and action rights over every report in the system.

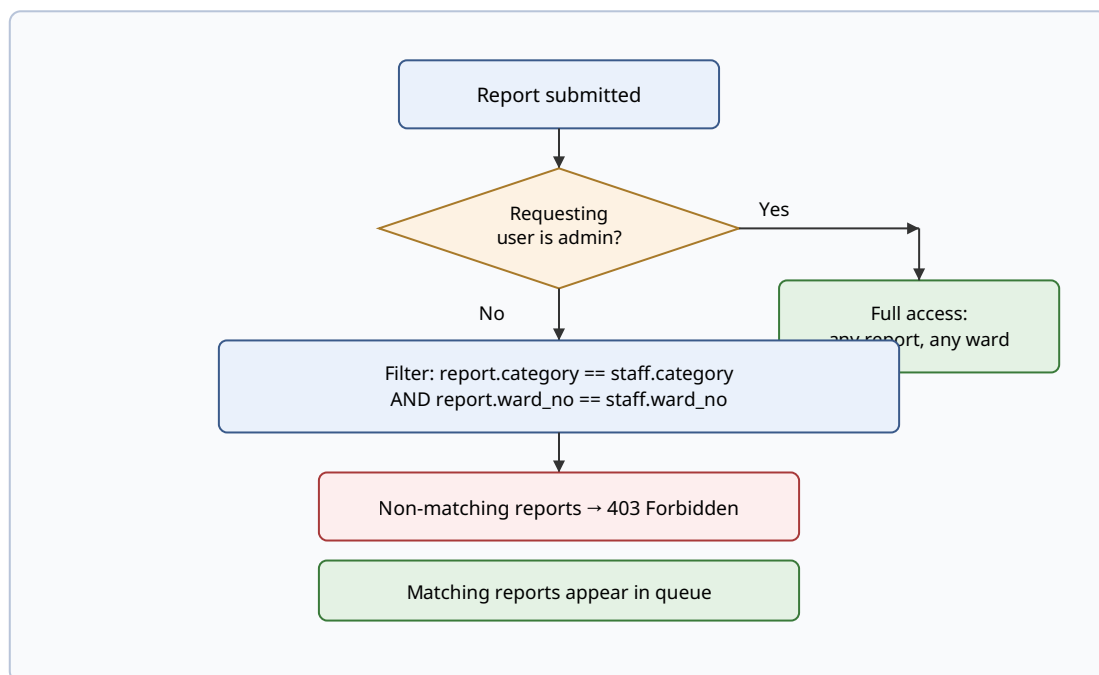


Figure 6.3 Category + Ward Auto-Routing & Admin Override Diagram

[SCREENSHOT]

Staff dashboard showing the queue of reports for the logged-in staff member's category and ward, with Verify / Reject / Progress / Complete actions

Figure 6.1 Staff Dashboard

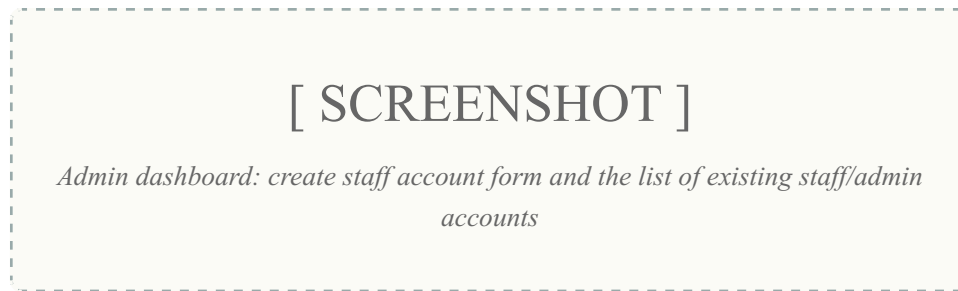


Figure 6.2 Admin Dashboard – Staff Management

Live Audit Trail Example

The table below is the genuine, unedited `status_logs` history returned by `GET /reports/{id}` for a report taken through its full lifecycle during testing of this project — submitted by a citizen, then verified, progressed, and completed by an admin account.

Table 6.1 Live Audit Log Entries (Sample Report)

Timestamp (UTC)	Action	Comment
2026-09-09 16:22:01	submitted	— (citizen submission, <code>staff_id</code> is NULL)
2026-09-09 16:23:46	verified	Confirmed pothole via photo, matches ward 5 road records.
2026-09-09 16:23:49	in_progress	Road crew scheduled for next week. (<code>expected_completion_date</code> set to 2026-09-20)
2026-09-09 16:23:51	completed	Pothole filled and resurfaced.

Figure 6.4 Live Audit Trail of a Resolved Report

This immutable, append-only log is what makes every status change on a report traceable to a specific staff member (or the citizen, for the initial submission), at a specific time, with an optional explanation — the same principle an audit log serves in the survey portals from Lab 1.

LAB 7: Data Protection

Purpose

The purpose of this lab is to document how the system protects user data during transmission, storage, and operation, and to describe its backup and privacy practices.

Sensitive Data Handled

Unlike a national voter or identity system, this project deliberately collects only what is needed to attribute and follow up on a civic complaint: full name, email, phone number, and a password. It does *not* collect citizenship numbers, national ID numbers, or any other high -sensitivity government identifiers, which keeps its data-protection surface intentionally small.

Protections Implemented

Table 7.1 Security Mechanisms Implemented

S.N	Mechanism	Purpose
1	bcrypt password hashing (passlib)	Passwords are never stored or logged in plain text
2	JWT authentication (HS256, expiring)	Stateless, tamper-evident session tokens
3	Role-based access control	Citizens, staff and admins each see and can do only what their role permits
4	Category + ward scoping	A staff account cannot act on reports outside its own assigned area
5	Pydantic request validation	Rejects malformed or out-of-range input before it reaches the database
6	SQLAlchemy ORM (parameterized queries)	Eliminates the SQL-injection surface of hand-built queries
7	Upload validation + Pillow re-encoding	Restricts file type/size and strips anything hidden in a malformed image
8	CORS allow-list	Only explicitly configured frontend origins may call the API
9	Fake-report lockout	3 reports rejected as false_report within 7 days → citizen blocked from submitting for 7 days
10	Immutable status-log audit trail	Every status change is permanently recorded with who, what, and when
11	Anonymity toggle	

		display_publicly=false hides the citizen's name from the public feed while staff/admin always see the real identity for accountability
--	--	--

Transport Security

During development the system was tested locally over plain HTTP. For deployment, HTTPS/TLS is provided automatically and free of charge by the hosting platform (Render terminates TLS at its edge for both the API and the static frontend), so no certificate management is required in the application itself.

Backup and Recovery Procedure

Because the database is hosted on Neon (managed PostgreSQL), backup relies on standard PostgreSQL tooling rather than a phpMyAdmin export:

Backup:

1. Run `pg_dump "$DATABASE_URL" -F c -f backup_YYYY-MM-DD.dump` to export a full logical backup of the database.
2. Separately back up the `backend/uploads/` folder, which holds the actual report photos referenced by `photo_path` in the database.
3. Store the `.env` file's secrets (`JWT_SECRET_KEY`, `DATABASE_URL`) in a secure secrets manager or password vault, never committed to version control.

Recovery:

1. Provision a fresh PostgreSQL database (or reuse the existing Neon project) and run `python -m app.database` to recreate the schema, or restore directly with `pg_restore`.
2. Restore the matching `uploads/` backup so existing `photo_path` values resolve correctly.
3. Log in as an admin account and spot-check a few reports to confirm the restored data is consistent before resuming normal operation.

Security and Privacy Policy

Security: only authenticated users can access protected resources; citizens can access only their own reports and profile; staff act only within their assigned category/ward; every status change is logged; passwords are never stored in plain text.

Privacy: the system collects only the information required for report attribution and follow-up (name, email, phone). A citizen who reports anonymously has their name withheld from the public feed — only staff/admin (who need it for follow-up) can see the real identity behind any report.

Deployment

Purpose

This section documents the intended deployment of the Local Civic Problem Reporting System to a publicly reachable environment.

Deployment Architecture

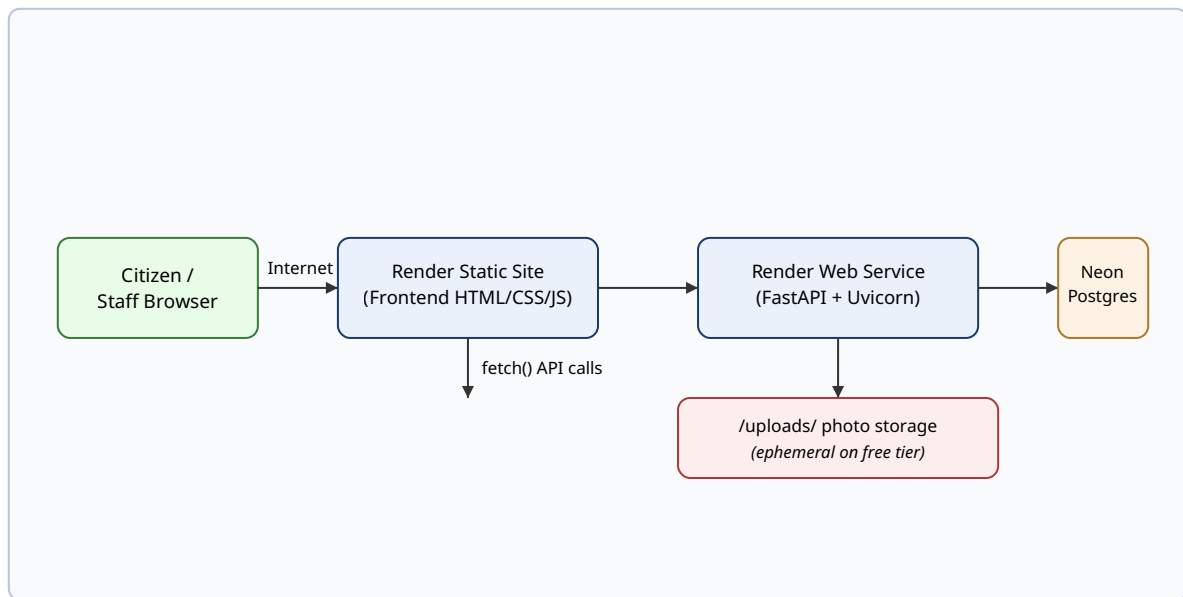


Figure 8.1 Deployment Architecture of the Civic Report System

Platform Configuration

Table 8.1 Deployment Platform Configuration

S.N	Component	Description
1	Hosting Platform	Render (free tier)
2	Backend Service	Render Web Service — build: <code>pip install -r requirements.txt</code> ; start: <code>uvicorn app.main:app --host 0.0.0.0 --port \$PORT</code>
3	Frontend	Render Static Site, serving <code>frontend/</code> directly
4	Database	Neon (managed PostgreSQL, already in use during development)
5	Environment Variables	<code>DATABASE_URL</code> , <code>JWT_SECRET_KEY</code> , <code>JWT_ALGORITHM</code> , <code>JWT_EXPIRE_MINUTES</code> , <code>UPLOAD_DIR</code> , <code>MAX_UPLOAD_MB</code> , <code>CORS_ORIGINS</code> — set in the Render dashboard, never committed to the repository

6	TLS/HTTPS	Provided automatically by Render at no extra cost
---	-----------	---

Known limitation: Render's free-tier web services use an ephemeral filesystem, so files saved to `backend/uploads/` do not persist across restarts or redeploys. For a class demonstration this is acceptable; a production deployment would move photo storage to an object store (e.g. S3-compatible storage) instead of local disk. Free-tier services also spin down after 15 minutes of inactivity and take roughly 30–50 seconds to wake on the next request.

Live URL: *[to be filled in after the Render deployment is completed — e.g. <https://civic-report-api.onrender.com> and <https://civic-report.onrender.com>]*

Testing

Purpose

Testing verifies that the system works correctly, that the complete citizen→staff→admin workflow operates smoothly, and that it resists common security issues.

Unit Testing

Table 9.1 Summary of Unit Test Cases

Test ID	Module	Description	Expected	Actual (Live)	Status
UT-01	Registration	Register with valid data	201, account created	201, account created	Pass
UT-02	Registration	Register with an email already used	400, rejected	400, "email already exists"	Pass
UT-03	Login	Login with valid credentials	200, JWT returned	200, JWT returned	Pass
UT-04	Login	Login with wrong password	401, generic error	401, generic error	Pass
UT-05	Report Submission	Submit report with valid photo	201, report stored	201, report stored	Pass
UT-06	Report Submission	Submit report with invalid category	400, rejected	400, rejected	Pass
UT-07	Report Submission	Submit report with a .txt file as photo	400, rejected	400, "Only JPG and PNG..."	Pass

System Testing

Table 9.2 Summary of System Test Cases

Test ID	Feature	Expected Result	Status
ST-01	Citizen registration & login	Account created, JWT issued on login	Pass
ST-02	Report submission with photo + GPS	Report stored with photo_path and coordinates	Pass
ST-03			Pass

	Public feed anonymity	display_publicly=false shows "Anonymous Citizen"; staff/admin still see the real name via the staff endpoint	
ST-04	Staff/Admin verification workflow	Status transitions submitted→verified→in_progress→completed, each logged	Pass
ST-05	Category + ward scoping	Non-admin staff blocked (403) from reports outside their own category/ward	Pass
ST-06	Audit logging	Every status change recorded in status_logs with timestamp and optional comment	Pass

End-to-End Workflow Test

The following was executed live, end-to-end, against the running system:

Table 9.3 End-to-End Workflow Test Results

Step	Action	Result	Status
1	Citizen registers	201 Created	Pass
2	Citizen logs in	200 OK, JWT issued	Pass
3	Citizen submits a report with a photo and GPS coordinates	201 Created, status = submitted	Pass
4	Report appears in the admin's report queue	Visible in GET /staff/reports	Pass
5	Admin verifies the report	status = verified, logged	Pass
6	Admin sets it in-progress with an expected completion date	status = in_progress, date stored, logged	Pass
7	Admin marks it completed	status = completed, logged	Pass
8	Public detail endpoint shows the full 4-step audit trail	All 4 status_logs entries returned in order	Pass

Challenges Faced

The purpose of this section is to describe real technical challenges encountered while building and verifying this project, and how each was resolved.

Table 10.1 Challenges Faced and Resolutions

S.N	Challenge	Resolution
1	A password-hashing library incompatibility caused every registration to fail at runtime with a misleading error ("password cannot be longer than 72 bytes") even for short passwords.	Root-caused it to an unpinned <code>bcrypt</code> dependency: <code>passlib==1.7.4</code> is incompatible with <code>bcrypt>=4.1</code> 's changed internal API, which broke <code>passlib</code> 's self-test on startup. Pinned <code>bcrypt==4.0.1</code> in <code>requirements.txt</code> and reinstalled, which fixed registration and login immediately — confirmed by re-running the same test that had failed.
2	Designing automatic report routing without a manual dispatch step	Instead of a ward→district escalation chain, gave every staff account a fixed (category, ward_no) pair and filtered the report queue query by an exact match, with admin accounts exempted from the filter.
3	Preventing a citizen's real identity from leaking through the "anonymous" toggle	Built two separate response shapes (<code>ReportOut</code> for the public feed, which derives <code>reporter_name</code> from <code>display_publicly</code> , versus <code>ReportStaffOut</code> , which always includes the real name/email/phone) so the anonymity rule is enforced by the schema itself, not by a check that could be forgotten in some other endpoint.
4	Stopping repeat false reporting without permanently banning a citizen for one mistake	Implemented a rolling window: only <code>false_report</code> rejections within the last 7 days count toward the 3-strike threshold, computed from the real <code>status_logs</code> timestamps rather than a simple ever-incrementing counter, so an old flag naturally falls out of the window.
5	Keeping uploaded photos small and safe without rejecting legitimate large phone-camera photos	Used <code>Pillow</code> to resize any image whose longest side exceeds 1600px and re-encode JPEGs at quality 85 before saving, while still enforcing a hard size cap and file-type allow-list ahead of that step.
6	CORS blocking the frontend during local development	Restricted <code>CORS_ORIGINS</code> to only the local dev server origin during development, then documented that this must be updated to the deployed frontend's real URL once hosted on Render.

Conclusion and Future Scope

Conclusion

The Local Civic Problem Reporting System successfully digitizes the reporting, verification, and resolution of local infrastructure problems through a secure web-based platform. It provides role-based access for citizens, category/ward-scoped staff, and administrators, and replaces an undocumented paper process with one that gives every report a photograph, an optional GPS location, a transparent status timeline, and a public feed that builds accountability into the system by default. The project met its objectives by implementing secure JWT-based authentication, a complete report lifecycle with an immutable audit trail, automatic category-and-ward routing, and an abuse-prevention mechanism for repeat false reporting.

Future Scope

Table 11.1 Future Scope

S.N	Enhancement	Description
1	SMS / Push Notifications	Notify citizens when their report changes status, for those without reliable access to check the web app.
2	Login Rate-Limiting & CAPTCHA	Add a failed-login lockout and a CAPTCHA challenge to harden the login endpoints against automated/bot abuse.
3	Object Storage for Photos	Move uploaded photos from local disk to S3-compatible object storage so they persist across redeloys on platforms with an ephemeral filesystem.
4	Analytics Dashboard	Ward/category-level dashboards showing report volume, average resolution time, and repeat-problem hotspots on a map.
5	Native Mobile App	A dedicated mobile client (or PWA) with offline photo capture and push notifications, rather than only a responsive web page.
6	Multi-language Support	Add Nepali-language UI alongside English to improve accessibility for a broader range of citizens.

References

1. Wikipedia — E-governance. [Online]. <https://en.wikipedia.org/wiki/E-governance>
2. Government of Nepal — Nagarik App. [Online]. <https://nagarikapp.gov.np>
3. Government of Nepal — Inland Revenue Department. [Online]. <https://ird.gov.np>
4. Government of Nepal — Department of Passports. [Online]. <https://nepalpassport.gov.np>
5. FastAPI Documentation. [Online]. <https://fastapi.tiangolo.com>
6. SQLAlchemy Documentation. [Online]. <https://docs.sqlalchemy.org>
7. Pydantic Documentation. [Online]. <https://docs.pydantic.dev>
8. Leaflet.js Documentation. [Online]. <https://leafletjs.com>
9. OpenStreetMap. [Online]. <https://www.openstreetmap.org>
10. Neon (Serverless Postgres) Documentation. [Online]. <https://neon.tech/docs>
11. Render Documentation. [Online]. <https://render.com/docs>
12. PyJWT / python-jose Documentation. [Online]. <https://python-jose.readthedocs.io>
13. OWASP Top Ten. [Online]. <https://owasp.org/www-project-top-ten/>